Department of Information and Communication Technology

PROJECT REPORT

on

IoT based Smart Parking and Monitoring System

Submitted by

Md. Ashikur Rhaman Ashik
ID: IT-24047
Department of Information and Communication Technology

Under the supervision of

Mr. Md. Anowar Kabir
Lecturer
Department of Information and Communication Technology

Date of Submission: _____

Abstract

Finding a free parking space wastes time and fuel, and manual gate and record keeping is slow and error-prone. This project presents an **IoT based Smart Parking and Monitoring System** that detects the occupancy of parking slots automatically, shows the status to drivers on site, records every vehicle entry and exit with time, and lets an administrator monitor the lot and control the entry gate from a web dashboard.

The system uses an **ESP32** microcontroller connected to eight infrared (IR) obstacle sensors, one for each slot of two zones (Zone A for cars and Zone B for buses). Red and green LEDs at the slots, a 16×2 I2C LCD and a buzzer give immediate local feedback, while a servo motor operates the entry gate. The ESP32 sends the sensor data, entry time, exit time and parking duration to the **Firebase Realtime Database** over WiFi. A responsive **web dashboard** built with HTML, CSS and JavaScript reads the same database in real time, shows live slot status, vehicle access records and date-wise history, and writes a single true/false value to open or close the gate. A login page restricts access to the dashboard.

The local indicators work even if the internet connection is lost, and the cloud records are recovered after a reboot, which makes the system reliable and inexpensive to build.

Keywords: IoT, ESP32, Smart Parking, Firebase, IR Sensor, Servo Motor, Real-time Monitoring.

Contents

Abstract 1

1 Introduction 3

1.1 Background . . . 3
1.2 Problem Statement . . . 3
1.3 Objectives . . . 3
1.4 Scope . . . 4
1.5 Organization of the Report . . . 4

2 System Overview 4

2.1 Architecture . . . 4
2.2 Main Features . . . 4

3 Tools and Components 5

3.1 Hardware Components . . . 5
3.2 Software Tools . . . 5
3.3 Component Description . . . 5

4 Hardware Design 6

4.1 Pin Connections . . . 6
4.2 Power Supply . . . 6

4.3 Circuit Diagram and Prototype 6

5 Software Design 6

5.1 Firebase Database Structure 6

5.2 ESP32 Firmware 6

5.3 Web Dashboard 9

5.4 Login Page 10

5.5 Screenshots of the Web Application 10

6 Working Principle 10

6.1 Vehicle Entry 10

6.2 Vehicle Exit 12

6.3 Gate Operation 12

6.4 Driver Information 12

7 Testing 12

8 Limitations and Future Work 13

8.1 Limitations 13

8.2 Future Work 13

9 Conclusion 13

References 14

A Firmware and Web File List 14

List of Figures

2.1 Block diagram of the proposed system 4

4.1 Circuit and connection diagram of the system (IR sensors, LED indicators and I2C LCD connected to the ESP32, with data sent to Firebase and the web dashboard) 7

4.2 Hardware components used in the prototype (ESP32, IR sensor, SG90 servo motor, I2C LCD, buzzer and battery) 7

4.3 Hardware prototype of the smart parking system 8

5.1 Realtime Database structure 8

5.2 Flowchart of slot monitoring in the ESP32 firmware 9

5.3 Login page 10

5.4 Dashboard with gate control, statistics and live slots 11

5.5 Live Monitoring view 11

5.6 Vehicle History (date wise) view 11

List of Tables

3.1 Hardware components 5
3.2 Software tools and libraries 5

4.1 IR sensor connections 6
4.2 LED, LCD, servo and buzzer connections 6

5.1 Data ownership 7
5.2 Dashboard views 10

7.1 Test cases 12

A.1 Project files 14

1. Introduction

1.1 Background

Rapid growth in the number of vehicles has made parking a daily problem in cities, campuses and commercial buildings. Drivers circle the lot looking for a free space, and operators rely on manual observation and paper registers to track entry and exit. The Internet of Things (IoT) allows low-cost sensors and microcontrollers to collect data from the physical world and publish it to the cloud, where it can be viewed from anywhere. Applying this idea to parking gives a system that is automatic, visible and traceable.

1.2 Problem Statement

Conventional parking lots have the following problems:

- Drivers cannot tell which slots are free without driving around the lot.
- Entry and exit times are written by hand, which is slow and may contain mistakes.
- Management cannot see the current occupancy from a remote location.
- The entry gate is operated manually.

1.3 Objectives

The main objectives of this project are:

1. To detect the occupancy of eight parking slots automatically using IR sensors.
2. To show slot status locally using red/green LEDs, an LCD and a buzzer.
3. To record the date, entry time, exit time and parking time of every vehicle.
4. To store the data in the cloud and show it live on a web dashboard.
5. To control the entry gate (servo motor) from the web dashboard.
6. To protect the dashboard with a login page.

1.4 Scope

The prototype covers one entry gate and two zones with four slots each: Zone A (car parking, slots A01–A04) and Zone B (bus parking, slots B01–B04). Vehicle number recognition, payment and mobile applications are outside the scope of this version and are listed as future work.

1.5 Organization of the Report

Chapter 2 describes the system overview. Chapter 3 lists the hardware and software tools. Chapter 4 presents the hardware design and connections. Chapter 5 explains the software design: firmware, database and web application. Chapter 6 describes the working principle. Chapter 7 covers testing. Chapter 8 discusses limitations and future work, and Chapter 9 concludes the report.

2. System Overview

2.1 Architecture

The system has three layers. The **device layer** (ESP32 with sensors and output devices) senses the lot and acts locally. The **cloud layer** (Firebase Realtime Database) stores the shared state. The **application layer** (web dashboard) displays the data and sends the gate command. The ESP32 writes slot and history data; the web page writes only the gate command. This separation keeps the records correct even when the browser is closed.

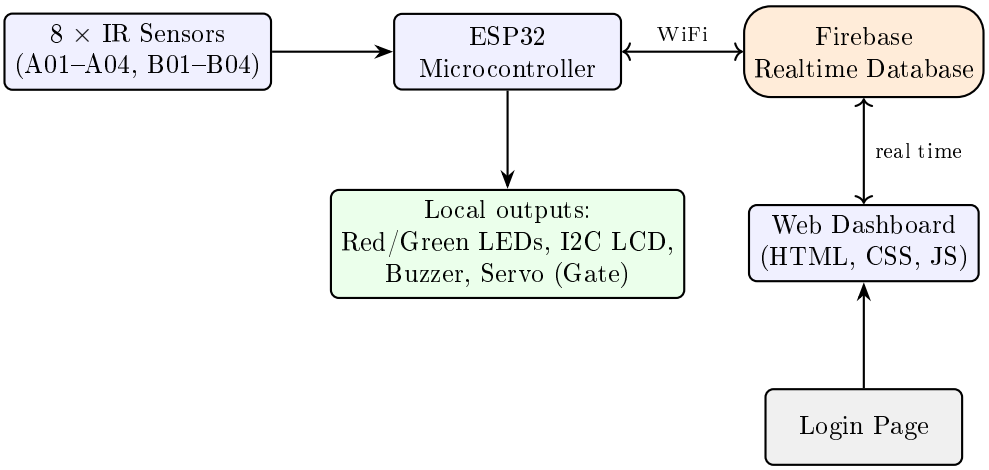


Figure 2.1: Block diagram of the proposed system

2.2 Main Features

- Live status of eight slots in two zones (green = available, red = occupied).
- Parking timer for each occupied slot.
- Automatic entry/exit recording with date, time and parking duration.
- Vehicles Inside counter and active session count.
- Date-wise vehicle history with From/To filter.
- Remote gate Open/Close control (Firebase value `true`/`false`).
- Local LED, LCD and buzzer feedback that works without internet.
- Light and dark theme and a responsive layout for phones.

3. Tools and Components

3.1 Hardware Components

Table 3.1: Hardware components

Qty	Component	Purpose
1	ESP32 development board	Main controller with built-in WiFi
8	IR obstacle sensor module	Detect a vehicle in each slot
1	Servo motor (SG90 type)	Opens and closes the entry gate
1	16×2 LCD with I2C backpack	Shows messages and free slots
1	Buzzer module (3-pin)	Audio feedback
5	Red LED	Slot occupied indicator
5	Green LED	Slot available indicator
10	220 Ω resistor	Current limiting for LEDs
1	5 V, 2 A power supply	Powers the board, servo and modules
–	Breadboard and jumper wires	Connections

3.2 Software Tools

Table 3.2: Software tools and libraries

Tool	Use
Arduino IDE	Programming and uploading the ESP32 firmware
Firebase ESP Client (Mobizt)	Reading and writing the Realtime Database from ESP32
ESP32Servo	Servo control on ESP32
LiquidCrystal I2C	LCD control over I2C
Firebase Realtime Database	Cloud storage and real-time synchronisation
HTML, CSS, JavaScript	Web dashboard and login page
Firebase JavaScript SDK	Real-time listeners in the browser

3.3 Component Description

ESP32. A dual-core microcontroller with WiFi and Bluetooth. It reads the sensors, drives the outputs and communicates with Firebase.

IR obstacle sensor. An infrared transmitter and receiver pair. When an object is in front of it the digital output goes LOW (active low). One sensor is placed at each slot.

Servo motor. Rotates to a commanded angle. In this project it is 0° for a closed gate and 90° for an open gate.

I2C LCD. A 16×2 character display that needs only two data lines (SDA and SCL).

Buzzer. Gives short beeps for events such as a vehicle entering or the gate moving.

Firebase Realtime Database. A cloud NoSQL JSON database that pushes changes to every connected client within a short time.

4. Hardware Design

4.1 Pin Connections

Eight IR sensors, ten LEDs, an LCD, a servo and a buzzer use almost all usable ESP32 pins. The four input-only pins (GPIO 34, 35, 36, 39) are used for sensors so that output-capable pins remain for the LEDs.

Table 4.1: IR sensor connections

Slot	GPIO	Slot	GPIO
A01	34	B01	32
A02	35	B02	33
A03	36	B03	25
A04	39	B04	26

Table 4.2: LED, LCD, servo and buzzer connections

Device	Signal	GPIO
LED pair A01	Green / Red	27 / 14
LED pair A02	Green / Red	13 / 16
LED pair A03	Green / Red	17 / 4
LED pair A04	Green / Red	5 / 23
LED pair B01	Green / Red	15 / 2
I2C LCD	SDA / SCL	21 / 22
Servo motor	Signal	18
Buzzer	Signal	19

Each LED is connected through a 220 Ω resistor to ground. The sensors, LCD and buzzer share the common ground with the ESP32. The LED pin table in the firmware can be changed if LEDs are fitted to different slots; a value of -1 means that a slot has no LED.

4.2 Power Supply

The servo motor draws a high current while moving. A stable 5 V, 2 A supply is recommended so that the ESP32 does not reset when the gate moves.

4.3 Circuit Diagram and Prototype

5. Software Design

5.1 Firebase Database Structure

5.2 ESP32 Firmware

The firmware performs the following tasks in a continuous loop.

1. **Sensor reading and debouncing.** Every sensor is read continuously. A change is accepted only after it has stayed stable for 1 second, which prevents false triggers from a passing hand

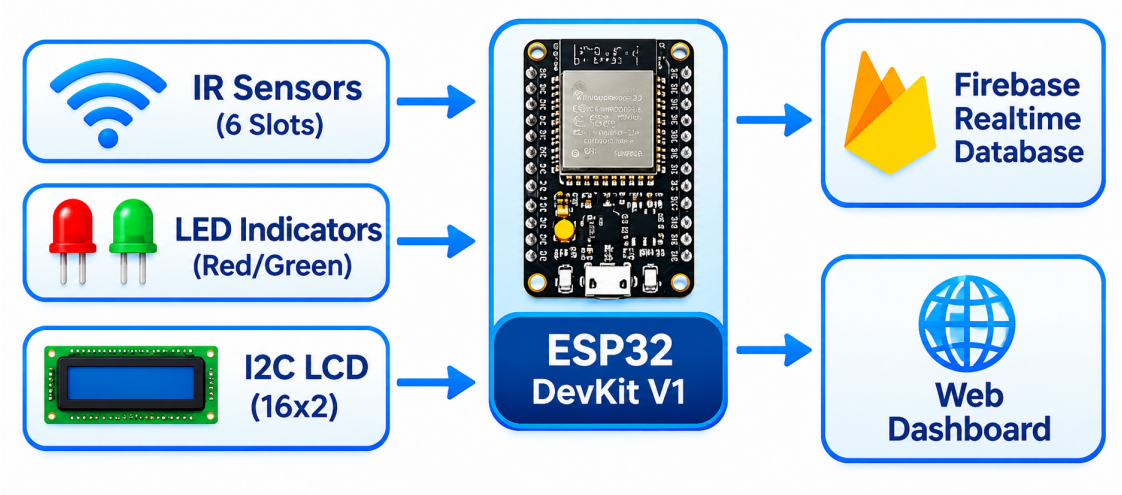


Figure 4.1: Circuit and connection diagram of the system (IR sensors, LED indicators and I2C LCD connected to the ESP32, with data sent to Firebase and the web dashboard)

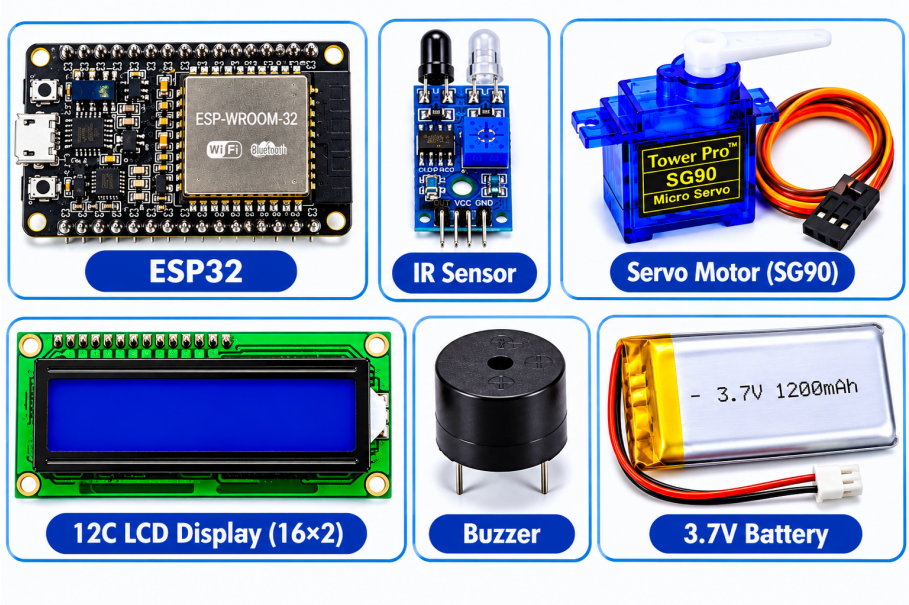


Figure 4.2: Hardware components used in the prototype (ESP32, IR sensor, SG90 servo motor, I2C LCD, buzzer and battery)

Table 5.1: Data ownership

Path	Written by	Read by
`gate/open`	Web dashboard	ESP32
`slots/*`	ESP32	Web dashboard
`history/*`	ESP32	Web dashboard
`status/vehiclesInside`	ESP32	Web dashboard



Figure 4.3: Hardware prototype of the smart parking system

```
root
|-- gate
|  `-- open : true / false
|-- slots
|  |-- A01 ... B04
|  |-- occupied : true / false
|  |-- entryDate, entryTime, entryEpoch
|  |-- exitDate, exitTime
|  `-- historyKey
|-- history
|  `-- <pushId>
|  |-- slotId, vehicleNo
|  |-- entryDate, entryTime, entryEpoch
|  |-- exitDate, exitTime, exitEpoch
|  `-- durationSeconds
`-- status
      `-- vehiclesInside : 0 ... 8
```

Figure 5.1: Realtime Database structure

or shadow.

2. **Local response.** When a stable change is detected the slot LED switches (green ↔ red), the LCD free-slot count is updated and the buzzer beeps once on entry. This part does not depend on the internet.

3. **Entry recording.** The firmware creates a new record under `history` with slot, date and entry time (Bangladesh time, UTC+6, obtained by NTP) and updates `slots` and `status/vehiclesInside`.

4. **Exit recording.** The same record is updated with exit date, exit time and the parking duration in seconds.

5. **Retry and recovery.** If a write fails it is retried after 3 seconds. After a reboot the firmware reads the stored history key of each slot, so an ongoing parking session continues without creating a duplicate record.

6. **Gate control.** Every second the value of `gate/open` is read. `true` moves the servo to 90° with one beep, `false` moves it to 0° with two beeps.

7. **LCD display.** Every 5 seconds the screen alternates between "Welcome / Entry Gate" and "Free Slot / ZoneA:n ZoneB:n".

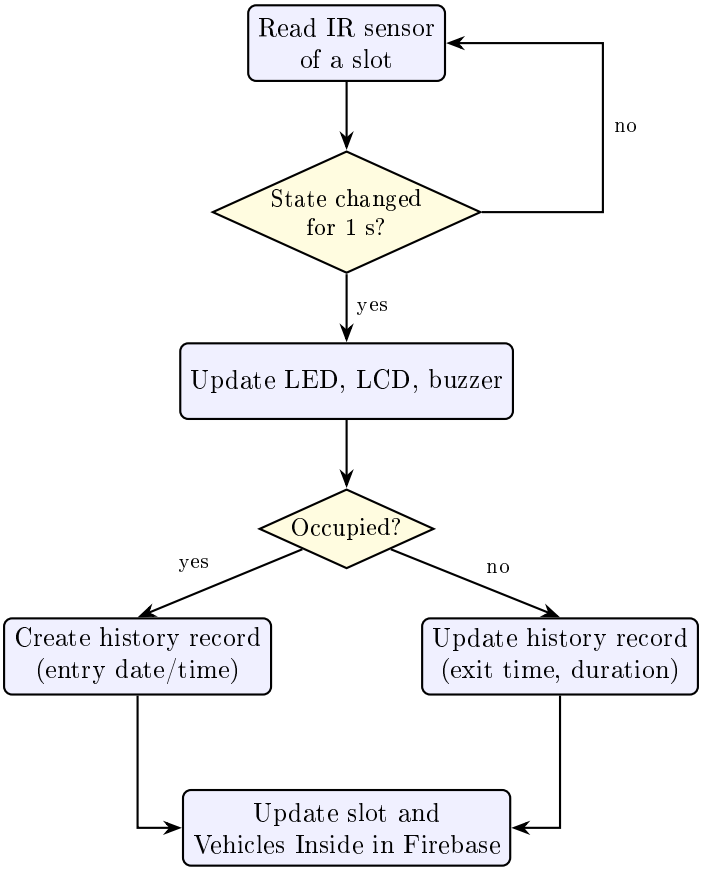


Figure 5.2: Flowchart of slot monitoring in the ESP32 firmware

5.3 Web Dashboard

The dashboard is a single-page application made of `index.html`, `style_1.css` and `script_1.js`. The Firebase SDK attaches real-time listeners, so the page updates without refreshing.

Table 5.2: Dashboard views

View	Content
Dashboard	Gate control card, statistics (Vehicles Inside, Sensor Slots Available, Active Parking Sessions), the eight live slots and the Vehicle Access table
Live Monitoring	Gate control and the live slot map for both zones
Vehicle History	Date-wise table (Date, Vehicle No, Zone, Entry Time, Exit Time, Parking Time) with From and To date filters

The page opens on the dashboard. The side menu opened with the nine-dot button switches between the views, so the history table is shown only when it is selected. The Open Gate button writes `true` to `gate/open` and Close Gate writes `false`. The parking timers are calculated in the browser and are never written to Firebase, which avoids unnecessary writes.

5.4 Login Page

`login.html` asks for a user name and a password that are set inside the file. If both are correct the page opens the dashboard link; otherwise it shows the message "Wrong User Name or Password. Again!" and clears the password field for a new attempt.

5.5 Screenshots of the Web Application

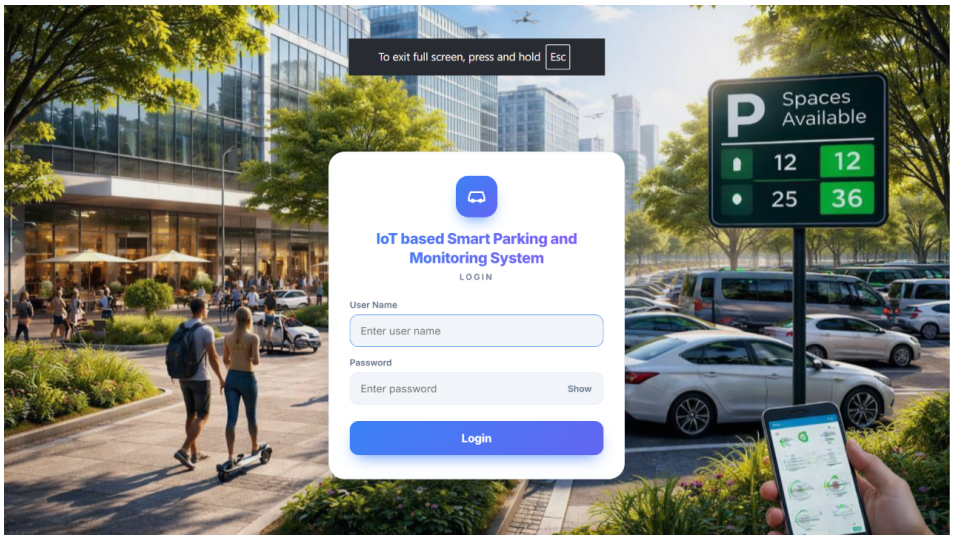


Figure 5.3: Login page

6. Working Principle

6.1 Vehicle Entry

1. The IR sensor of the slot detects the vehicle and its output goes LOW.
2. After the 1-second stability check the ESP32 turns the slot LED red, plays one beep and updates the free-slot count on the LCD.
3. The ESP32 stores the entry date and time in Firebase and increases `vehiclesInside`.
4. The dashboard turns the slot red, starts the "Parked" timer and adds a row to the Vehicle

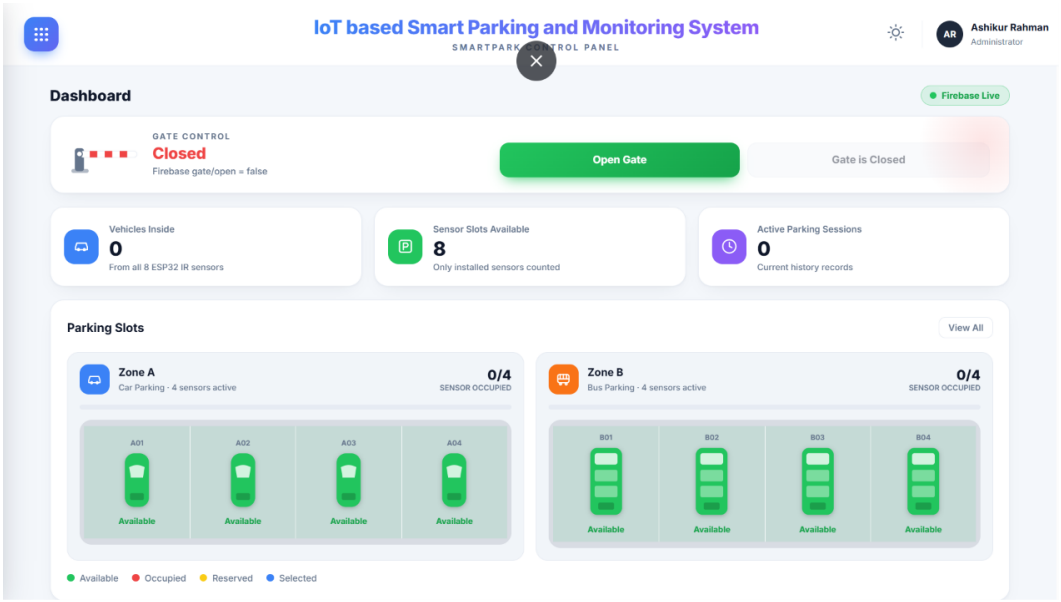


Figure 5.4: Dashboard with gate control, statistics and live slots

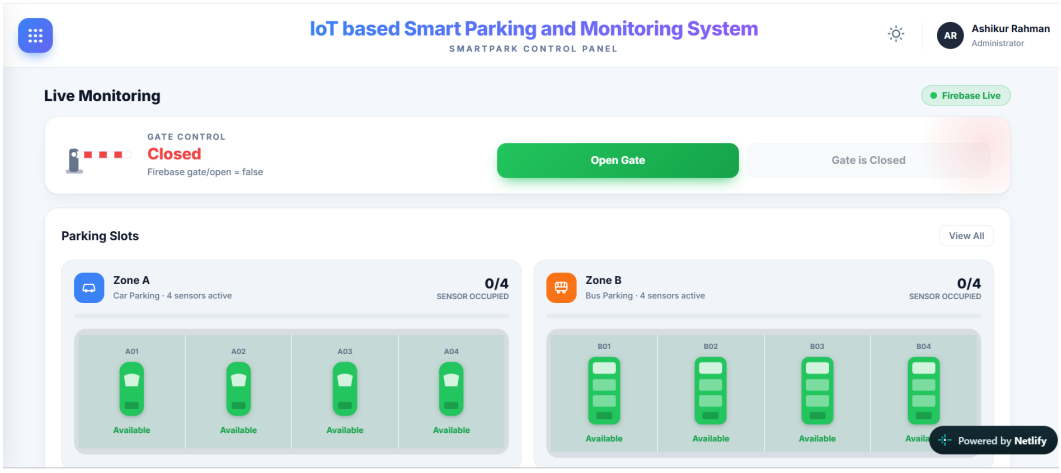


Figure 5.5: Live Monitoring view

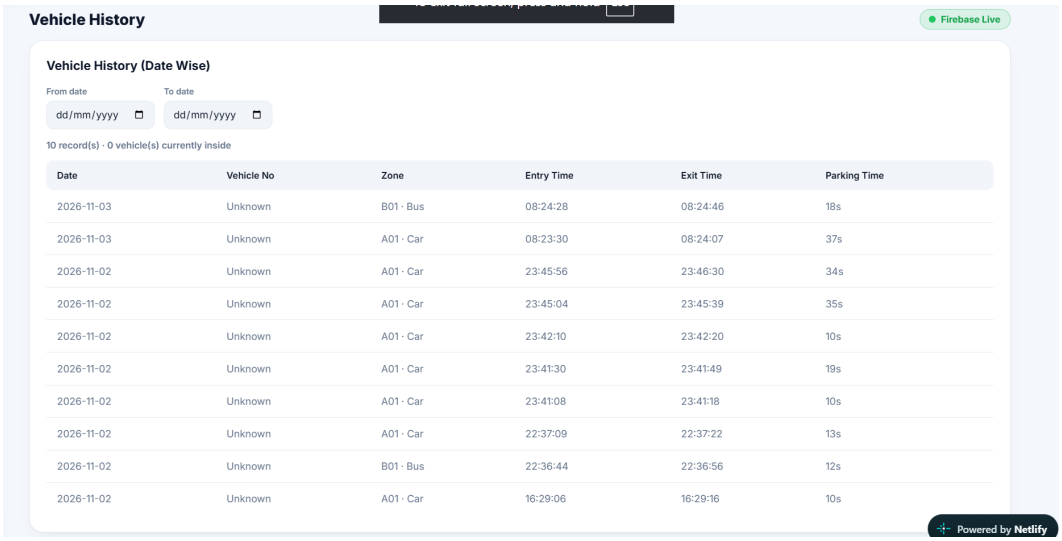


Figure 5.6: Vehicle History (date wise) view

Access table.

6.2 Vehicle Exit

1. The sensor output returns HIGH and, after the stability check, the LED turns green.
2. The ESP32 writes the exit time and the parking duration to the same history record and decreases `vehiclesInside`.
3. The dashboard shows the slot as available and fills the Exit Time and Parking Time columns.

6.3 Gate Operation

The administrator logs in and presses *Open Gate*. The value `true` is written to `gate/open`; the ESP32 reads it within one second, turns the servo to 90° and beeps once. Pressing *Close Gate* writes `false` and the gate closes with two beeps.

6.4 Driver Information

At the gate the LCD alternately shows a welcome message and the number of free slots in Zone A and Zone B. The red and green LEDs let a driver see from a distance which slots are free.

7. Testing

The system is tested part by part and then as a whole. Table 7.1 lists the test cases and the behaviour expected from the implemented design. The observed results should be recorded during the final demonstration.

Table 7.1: Test cases

#	**Test**	**Procedure**	**Expected result**
1	Sensor detection	Place an object in front of a slot sensor	Serial Monitor prints the slot as OCCUPIED; LED turns red
2	Debounce	Pass a hand quickly in front of the sensor	No change in LED or database
3	Entry record	Keep the object for more than 1 second	New record appears in `history` with entry date and time
4	Exit record	Remove the object	Exit time and duration are written to the same record
5	Vehicles Inside	Occupy two slots	`status/vehiclesInside` equals 2 and the dashboard shows 2
6	LCD	Observe the display for 10 seconds	Screens alternate every 5 seconds and the free-slot count follows the sensors
7	Buzzer	Occupy a slot and operate the gate	One beep on entry; one beep on open and two beeps on close
8	Gate open	Press Open Gate on the dashboard	`gate/open` becomes `true`; servo moves to 90°

#	Test	Procedure	Expected result
9	Gate close	Press Close Gate	`gate/open` becomes `false`; servo returns to 0°
10	History filter	Select a From/To date range	Only records of the selected dates are listed
11	Login	Enter wrong then correct credentials	Wrong: "Again!" message; correct: dashboard opens
12	Offline behaviour	Switch off the WiFi router	LED, LCD and buzzer still work; cloud records resume when WiFi returns
13	Reboot recovery	Reset the ESP32 with a vehicle parked	No duplicate history record is created

8. Limitations and Future Work

8.1 Limitations

- The vehicle number is not read, so `vehicleNo` is stored as "Unknown".
- The login credentials are stored in the page itself. This only prevents casual access and is not a secure authentication method.
- The database rules used for the prototype allow open read and write access. Anyone who knows the database address could read or change data.
- IR sensors can be affected by strong sunlight or reflective surfaces.
- The ESP32 polls the gate value once per second, so there is a small delay between the button press and the gate movement.
- The prototype uses almost all usable ESP32 pins; more slots would need an I/O expander.

8.2 Future Work

- Add a camera with number plate recognition or an RFID reader to identify vehicles.
- Use Firebase Authentication and strict database rules for real security.
- Develop a mobile application with slot reservation and notifications.
- Add parking fee calculation and online payment.
- Use ultrasonic or magnetic sensors for higher accuracy.
- Add an automatic exit gate and a larger multi-level parking layout.

9. Conclusion

The IoT based Smart Parking and Monitoring System was designed to give automatic slot detection, local indication, cloud record keeping and remote gate control at low cost. The ESP32 handles sensing and local feedback, Firebase keeps a single shared copy of the data, and the web dashboard presents live status and history to the administrator. Because the LEDs, LCD and buzzer are driven directly from the sensors, drivers still receive guidance if the network is down, and the recovery logic prevents duplicate records after a restart.

The project shows that a small number of inexpensive components can solve a real parking management problem. With stronger authentication and vehicle identification it can be extended into a complete commercial parking solution.

Bibliography

[1] Espressif Systems, *ESP32 Series Datasheet*. https://www.espressif.com

[2] Google, *Firebase Realtime Database Documentation*. https://firebase.google.com/docs/database

[3] Mobizt, *Firebase Arduino Client Library for ESP8266 and ESP32*. https://github.com/mobizt/Firebase-ESP-Client

[4] Arduino, *Arduino IDE and Language Reference*. https://www.arduino.cc

[5] Mozilla Developer Network, *Web Technology for Developers: HTML, CSS and JavaScript*. https://developer.mozilla.org

A. Firmware and Web File List

Table A.1: Project files

File	Description
`smartpark_esp32.ino`	ESP32 firmware: sensors, LEDs, LCD, buzzer, servo and Firebase
`index.html`	Dashboard page structure
`style_1.css`	Dashboard styling, light and dark theme
`script_1.js`	Dashboard logic and Firebase listeners
`login.html`	Login page
`smartpark_rules.json`	Database rules used for the prototype
`smartpark_database.json`	Initial database data